

Reviewer Pack — Plethysm Coefficient Gap in Symmetric Squares of Permutation...

Entry 661b05479d3e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 08:36:18 UTC

Plethysm Coefficient Gap in Symmetric Squares of Permutation Representations for Random CNFs

Entry ID: 661b05479d3e Verdict: INCONCLUSIVE Recorded: 2026-05-09 08:36:18 UTC

1. Conjecture

Field A (mathematical branch): Plethysm Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For random CNF formulas with m clauses and n variables, let $\rho(f)$ denote the plethysm coefficient of the symmetric square of the permutation representation of f 's clause incidence matrix. Then $\rho(f) \geq \Omega(n^{\{1.5\}})$ for satisfiable formulas, while $\rho(f) \leq O(n^{\{0.5\}})$ for unsatisfiable formulas.

Rationale (proposer's reasoning):

Plethysm coefficients capture symmetric power decompositions of permutation representations, which may reveal structural differences between satisfiable and unsatisfiable CNFs. The symmetric square's decomposition could expose algebraic complexity gaps tied to circuit size.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ac6c7b9e3faa7e9b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False
    return True

def generate_primes(n):
    primes = []
    candidate = 2
    while len(primes) < n:
        if is_prime(candidate):
            primes.append(candidate)
        candidate += 1
    return primes

def gaussian_elimination(matrix, b):
    n = len(matrix)
    for i in range(n):
        max_row = i
        for j in range(i + 1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, n):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - sum(matrix[i][j] * x[j] for j in range(i + 1, n))) / matrix[i][i]
    return x
```

```

def matrix_multiply(A, B):
    m, k, n = len(A), len(B), len(B[0])
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for l in range(k):
                C[i][j] += A[i][l] * B[l][j]
    return C

def plethysm_coefficient(matrix):
    n = len(matrix)
    identity = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    symmetric_square = matrix_multiply(matrix, matrix)
    det = 1
    for i in range(n):
        det *= symmetric_square[i][i]
    return det

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    m = 2 * n
    clause_set = set()

    def generate_clause():
        variables = [random.randint(1, n) for _ in range(3)]
        if variables not in clause_set and variables[::-1] not in clause_set:
            clause_set.add(tuple(sorted(variables)))
        return variables

    clauses = [generate_clause() for _ in range(m)]

    if not all(candidates):
        return {
            "metric_name": "plethysm_coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    clause_matrix = [[0] * n for _ in range(m)]
    for i, clause in enumerate(candidates):
        for var in clause:
            clause_matrix[i][var - 1] = 1

    plethysm_coeff = plethysm_coefficient(clause_matrix)

    if len(clause_set) == m / 2: # Assuming half of the clauses are satisfiable
        return {
            "metric_name": "plethysm_coefficient",
            "metric_value": plethysm_coeff,
            "instances_tested": 1,
            "conjecture_holds": plethysm_coeff >= n ** 1.5,
            "counterexample": ""
        }

```

```

    }
else:
    return {
        "metric_name": "plethysm_coefficient",
        "metric_value": plethysm_coeff,
        "instances_tested": 1,
        "conjecture_holds": plethysm_coeff <= n ** 0.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or generate_primes(30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(metric_values) / len(metric_values)} std=0.0 support_f")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(metric_values) / len(metric_values)} std={math.sqrt(su")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con")
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_988224e6.py", line 124, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_988224e6.py", line 84, in run_trial
    clauses = [generate_clause() for _ in range(m)]
               ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_988224e6.py", line 80, in generate_clau
    if variables not in clause_set and variables[::-1] not in clause_set:
    ~~~~~
TypeError: unhashable type: 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to unhashable list type, preventing data collection | next: Fix list hashing issue in generate_clause() by using tuples instead of lists

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111308
2	propose	ollama_remote	qwen3:8b	0	0	28608
3	preregistration	ollama_remote	qwen3:8b	0	0	24192
4	novelty	ollama_remote	qwen3:8b	0	0	20691
5	novelty	ollama_remote	qwen3:8b	0	0	14596
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14940
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13111
8	judge	ollama_remote	qwen3:8b	0	0	15291

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 242737 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/661b05479d3e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/661b05479d3e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/661b05479d3e.tar.gz (if generated)